Wprowadzenie do programowania w języku Python

Sesja 3: Dane liczbowe i przykłady wykorzystania

dr inż. Leszek Tarkowski

Plan sesji

1. Przetwarzanie danych liczbowych
2. Studium przypadku: kto przeżył katastrofę Titanica?
3. Podsumowanie i dalsze kroki

Dziś zamieniamy liczby z `titanic3.csv` na statystyki i wykresy: narzędziami z poprzednich sesji oraz dwiema bibliotekami, NumPy i matplotlib.

Sesja zdalna (Clouda). Przed startem: udostępnianie ekranu z VS Code, powiększona czcionka edytora i terminala, środowisko z numpy i matplotlib (`uv sync` w folderze code albo `python -m pip install numpy matplotlib`).

Przypomnienie

```
from passenger_list import DATA_FILE, PassengerList

passengers = PassengerList.from_csv_file(DATA_FILE)
len(passengers)                      # 1309
f"{passengers.survival_rate():.1%}"  # '38.2%'

for k in (1, 2, 3):
    group = passengers.filter(lambda p: p.pclass == k)
    print(f"klasa {k}: {group.survival_rate():.1%}")
```

- `Passenger` to `@dataclass`, `PassengerList` zawiera 1309 takich obiektów.
- Dziś pytamy o **liczby**: wiek, opłaty, odsetki, i pokazujemy je na wykresach.

3.1 Przetwarzanie danych liczbowych

Liczby w komputerze: przypomnienie

```
2 ** 100                # 1267650600228229401496703205376
0.1 + 0.2               # 0.30000000000000004
1.1 + 2.2               # 3.3000000000000003
round(2.675, 2)         # 2.67
round(2.5), round(3.5)  # (2, 4)  zaokrąglenie bankierskie
```

- `float` to liczba binarna IEEE 754: ok. 15–16 cyfr znaczących, błąd względny rzędu 10^{-1}.
- W pomiarach i statystyce taki błąd jest bez znaczenia.
- W księgowości grosz to grosz: suma faktury musi się zgadzać co do grosza.

Od Pythona 3.12 sum() dla float stosuje sumowanie z kompensacją, więc sum([0.1] * 10) daje dokładnie 1.0; dodawanie w pętli (suma += 0.1) daje 0.9999999999999999. To pokazujemy w VS Code.

Pieniądze: `decimal.Decimal`

```
from decimal import Decimal

Decimal("0.1") * 3                  # Decimal('0.3')
sum([Decimal("0.10")] * 10)         # Decimal('1.00')
Decimal("19.99") + Decimal("0.01")  # Decimal('20.00')
```

`Decimal` przechowuje liczbę **dziesiętnie**: 0,1 jest zapisane dokładnie.

```
Decimal(0.1)
# Decimal('0.1000000000000000055511151231257827021181583404541015625')
```

`Decimal` tworzymy z napisu, nie z `float`: liczba zmiennoprzecinkowa jest już przybliżona.

Zaokrąglanie do groszy: `quantize`

```python
from decimal import Decimal, ROUND_HALF_UP

CENT = Decimal("0.01")
amount = Decimal("2.665")

amount.quantize(CENT)                              # Decimal('2.66')
amount.quantize(CENT, rounding=ROUND_HALF_UP)      # Decimal('2.67')
```

- Domyślnie `Decimal` też zaokrągla bankiersko (do parzystej).
- `ROUND_HALF_UP` to zaokrąglenie „szkolne”: 0,5 grosza i więcej w górę. Tak zaokrągla się kwoty podatku na fakturze.

Funkcja `vat()`

```python
from decimal import Decimal, ROUND_HALF_UP

CENT = Decimal("0.01")

def vat(net: Decimal, rate: Decimal = Decimal("0.23")) -> Decimal:
    """Kwota VAT zaokrąglona do pełnych groszy."""
    return (net * rate).quantize(CENT, rounding=ROUND_HALF_UP)

net = Decimal("19.99")
tax = vat(net)                 # Decimal('4.60')   (dokładnie: 4.5977)
net + tax                      # Decimal('24.59')
f"{net + tax:,.2f} zł"         # '24.59 zł'
```

Stawka jako Decimal(“0.23”), nie 0.23: Decimal(“19.99”) * 0.23 zgłasza TypeError (unsupported operand type(s) for *: ‘decimal.Decimal’ and ‘float’). To dobrze: chroni przed cichym przybliżeniem.

`float` czy `Decimal`?

	`float`	`Decimal`
Zapis	dwójkowy, sprzętowy	dziesiętny, programowy
Dokładność 0,1	przybliżona	dokładna
Szybkość	bardzo duża	wielokrotnie mniejsza

	`float`	`Decimal`
Zastosowanie	pomiary, statystyka, nauka, wykresy	kwoty, podatki, rozliczenia

Opłaty z Titanica analizujemy jako `float`: liczymy średnie i rysujemy rozkłady, a nie wystawiamy faktury.

Statystyki opisowe: `statistics`

```
import statistics as st

ages = [p.age for p in passengers if p.age is not None]

len(ages)                    # 1046
st.mean(ages)                # 29.88...
st.median(ages)              # 28.0
st.stdev(ages)               # 14.41...
st.quantiles(ages, n=4)      # [21.0, 28.0, 39.0]
min(ages), max(ages)         # (0.17, 80.0)
```

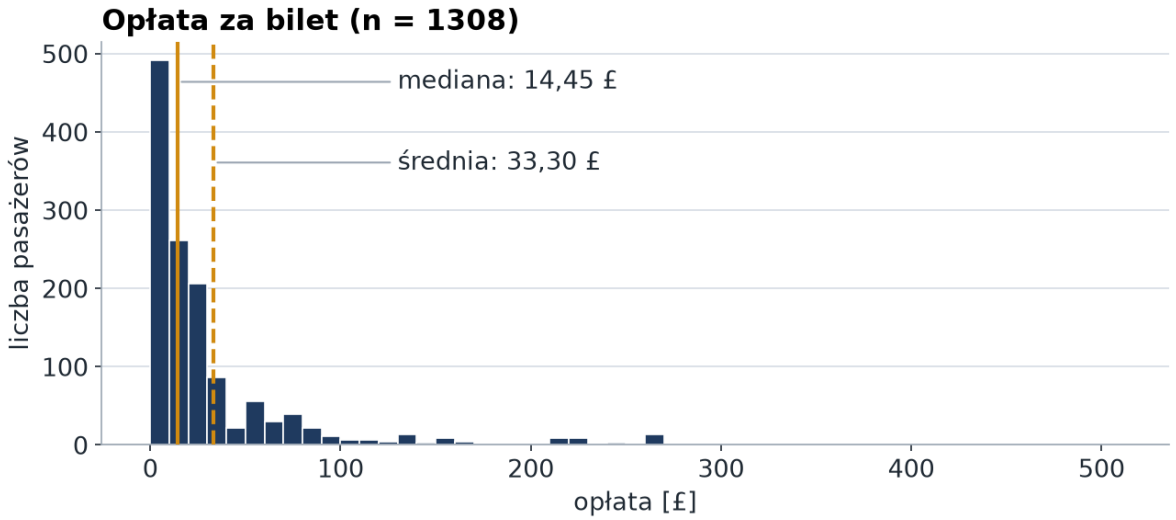
Braki (`None`) trzeba odfiltrować: `st.mean([29.0, None])` zgłasza `TypeError`.

stdev to odchylenie z próby (dzielnik n-1), pstdev z populacji (dzielnik n); przy n = 1046 różnica w trzeciej cyfrze znaczącej (14,413 vs 14,407).

Co mówią te liczby?

Miara	Wiek	Znaczenie
średnia	29,9	suma podzielona przez liczbę
mediana	28,0	połowa pasażerów młodsza, połowa starsza
odchylenie standardowe	14,4	typowa odległość od średniej
kwartyle	21 / 28 / 39	granice ćwiartek: połowa pasażerów miała 21–39 lat
min / max	0,17 / 80	najmłodszy miał 2 miesiące

Średnia a mediana



Kilka biletów po 512 £ podnosi średnią do 33,30 £, a typowy pasażer zapłacił 14,45 £.

Rozkład silnie prawoskośny: 75% pasażerów zapłaciło mniej niż 31,28 £. Ten sam efekt dają płace, ceny mieszkań, wartości zamówień. 17 biletów ma opłatę 0: to nie brak danych, tylko bilety darmowe.

Statystyki w grupach

```
for k in (1, 2, 3):
    group = passengers.filter(lambda p: p.pclass == k)
    fares = [p.fare for p in group if p.fare is not None]
    ages = [p.age for p in group if p.age is not None]
    print(k, len(group), st.median(fares), round(st.mean(ages), 1))
```

Klasa	Pasażerów	Mediana opłaty [£]	Średni wiek
1	323	60,00	39,2
2	277	15,05	29,5
3	709	8,05	24,8

Pokaz w VS Code

- Pułapki `float`: dodawanie 0,1 w pętli, `round(2.675, 2)`.

- `Decimal` z napisu i z `float`; `quantize` z `ROUND_HALF_UP`.
- Funkcja `vat()`; `Decimal("19.99") * 0.23` → `TypeError`.
- `statistics` na wieku i opłatach pasażerów; średnia a mediana.
- Statystyki w podziale na klasy przez `filter()`.

Po co NumPy?

Lista w Pythonie jest uniwersalna:

- może zawierać obiekty różnych typów,
- każdy element to osobny obiekt w pamięci,
- pętlę wykonuje interpreter, krok po kroku.

Tablica NumPy (`ndarray`) jest wyspecjalizowana:

- wszystkie elementy mają **ten sam typ** i leżą w pamięci jeden za drugim,
- operacje na całej tablicy wykonuje skompilowany kod w C, bez pętli w Pythonie.

NumPy jest podstawą prawie całego ekosystemu danych w Pythonie: pandas, matplotlib, scikit-learn, SciPy.

Tablica `ndarray`

```
import numpy as np

# wiek pięciu pierwszych pasażerów
w = np.array([29.0, 0.92, 2.0, 30.0, 25.0])

w.dtype          # dtype('float64')
w.shape          # (5,)
w * 12           # array([348.  ,  11.04,  24.  , 360.  , 300.  ])
w.mean()         # np.float64(17.384)
w.max()          # np.float64(30.0)
```

Działanie na tablicy dotyczy **każdego elementu**: `w * 12` to wiek w miesiącach.

Jeden typ dla całej tablicy

```
np.array([1, 2]).dtype          # dtype('int64')
np.array([1, 2.5]).dtype        # dtype('float64')
np.array([1, 2.5, "a"])         # array(['1', '2.5', 'a'], dtype='<U32')
```

- NumPy dobiera jeden typ, w którym zmieszczą się wszystkie elementy.
- Napis w tablicy liczb zamienia **wszystko** w napisy: liczyć się już nie da.

Dane do NumPy przygotowujemy wcześniej: konwersja typów i braki jak w sesji 2.

Operacje wektorowe

Lista: pętla

```
in_months = [x * 12 for x in ages]
adults = [x for x in ages if x >= 18]
```

Tablica: jedno wyrażenie

```
age_array = np.array(ages)   # 1046 wartości
age_array * 12               # cała tablica razy 12
age_array >= 18              # array([ True, False, False, ...])
age_array[age_array >= 18]   # tylko dorośli
```

Maski logiczne

```
children = age_array < 18    # tablica True/False, 1046 elementów
children.sum()               # np.int64(154)   True liczy się jako 1
age_array[children].mean()   # np.float64(9.10...)

age_array[(age_array >= 18) & (age_array < 60)].size  # 852
```

- **Maska** to tablica wartości logicznych tej samej długości co dane.
- `arr[mask]` wybiera elementy, dla których maska ma `True`.
- Warunki łączymy operatorami `&` (i), `|` (lub), `~` (nie), każdy warunek w nawiasie.

`and`/`or` nie działają na tablicach: Python pyta o prawdziwość całej tablicy i zgłasza ValueError (“truth value of an array is ambiguous”). Nawiasy są potrzebne, bo & wiąże silniej niż porównania.

Braki danych: `nan`

```
fares = np.array([p.fare if p.fare is not None else np.nan
                  for p in passengers])

np.isnan(fares).sum()       # np.int64(1)
fares.mean()                # np.float64(nan)
np.nanmean(fares)           # np.float64(33.295...)
np.nanmedian(fares)         # np.float64(14.4542)
```

- `None` nie pasuje do tablicy liczb; brak zapisujemy jako `np.nan` (*not a number*).
- `nan` „zaraża” wynik: każde działanie z `nan` daje `nan`.
- Funkcje `np.nanmean`, `np.nanmedian`, `np.nanmax` pomijają braki.

Ile zyskujemy?

```
import timeit

number_list = list(range(1_000_000))
arr = np.arange(1_000_000)

timeit.timeit(lambda: sum(x * x for x in number_list), number=10)
timeit.timeit(lambda: (arr * arr).sum(), number=10)
```

Wersja	Czas jednego przebiegu
lista + pętla	ok. 18 ms
NumPy	ok. 0,4 ms

Ten sam wynik, kilkadziesiąt razy szybciej. Dla 1309 pasażerów różnicy nie odczujemy; dla milionów wierszy tak.

Czasy zależą od komputera; zmierzyć na żywo. Suma kwadratów 0..999999 to ok. $3{,}3 \cdot 10\hat{}17$, mieści się w int64 (do ok. $9{,}2 \cdot 10\hat{}18$). Przy większym zakresie NumPy po cichu przepełniłby int64, a int Pythona nie: szybkość ma cenę.

Dane dwuwymiarowe

```
data = np.loadtxt("data/pulsar.csv", delimiter=",")

data.shape            # (80, 300): 80 profili po 300 próbek
data[0]               # pierwszy profil: 300 liczb
data[:, 114]          # próbka nr 114 we wszystkich profilach: 80 liczb
data.max(axis=1)      # maksimum każdego profilu: 80 liczb
data.mean(axis=0)     # profil uśredniony: 300 liczb
```

- `np.loadtxt` wczytuje cały plik liczb jednym wywołaniem.
- `axis=0` liczy „w dół” kolumn, `axis=1` „wzdłuż” wierszy.

Dane: 80 kolejnych impulsów pulsara CP 1919 (okres 1,337 s), zdigitalizowane z pracy doktorskiej H. D. Crafta (1970). Uśrednienie wielu impulsów to standardowa technika radioastronomii: szum maleje jak $1/\sqrt{N}$, a stabilny profil pulsara zostaje.

Pokaz w VS Code

- Lista a tablica: `np.array`, `dtype`, `shape`, działania na całej tablicy.
- Wiek pasażerów jako tablica; maski: dzieci, dorośli, seniorzy.
- Opłaty z `np.nan`; `mean` a `nanmean`.
- `timeit`: pętla a NumPy.
- `np.loadtxt` na danych pulsara; `axis=0` i `axis=1`.

matplotlib

```
import matplotlib.pyplot as plt
```

- Najstarsza i najpowszechniejsza biblioteka wykresów w Pythonie.
- Wykres do okna, do pliku PNG lub PDF, do notatnika.
- Na niej opierają się inne narzędzia: pandas, seaborn.

Dwa sposoby pracy: **interfejs `pyplot`** i **interfejs obiektowy**.

Dwa interfejsy

`pyplot`: funkcje działające na „bieżącym” wykresie

```
plt.plot(x, y)
plt.title("Profil pulsara")
plt.show()
```

Obiektowy: metody konkretnych obiektów

```
fig, ax = plt.subplots()
ax.plot(x, y)
ax.set_title("Profil pulsara")
plt.show()
```

Używamy interfejsu obiektowego: `fig` i `ax` to obiekty, a rysowanie to wywoływanie ich metod, jak w sesji 2.

Anatomia wykresu

Obiekt	Co to jest	Przykładowe metody
`Figure`	cała „kartka”	`fig.savefig()`, `fig.tight_layout()`
`Axes`	jeden układ współrzędnych z danymi	`ax.plot()`, `ax.bar()`, `ax.set_title()`
`Axis`	pojedyncza oś: zakres, podziałka	`ax.set_xlim()`, `ax.set_xticks()`

```
fig, ax = plt.subplots()                  # jedna figura, jeden układ
fig, (ax1, ax2) = plt.subplots(1, 2)      # dwa układy obok siebie
fig, ax = plt.subplots(figsize=(10, 4))   # rozmiar w calach
```

Nazwa Axes (układ współrzędnych) myli się z Axis (oś): jedno Axes ma dwa obiekty Axis.

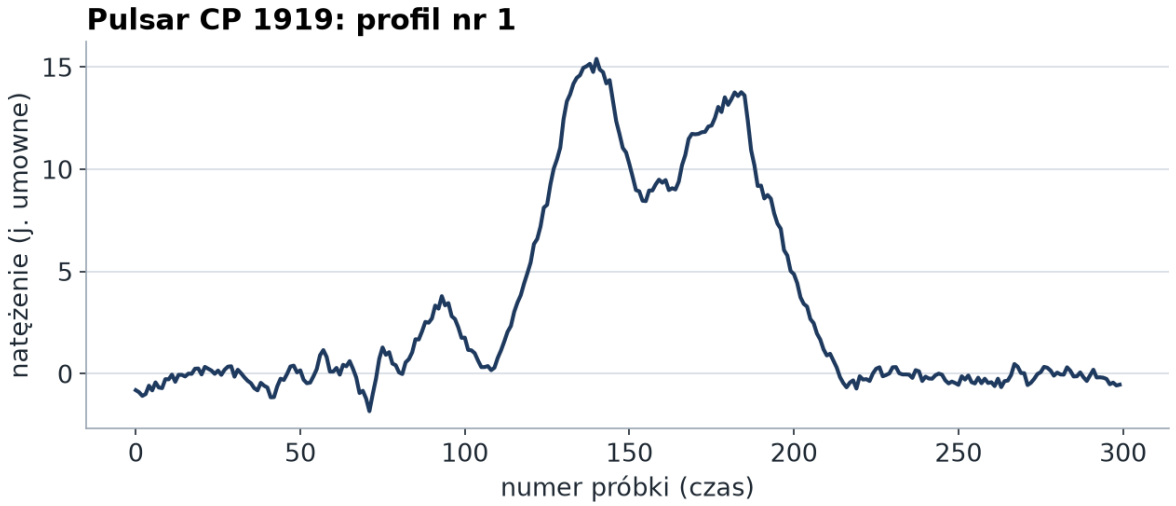
Pierwszy wykres

```
data = np.loadtxt("data/pulsar.csv", delimiter=",")

fig, ax = plt.subplots(figsize=(10, 3.6))
ax.plot(np.arange(300), data[0], color="#1F3A5F", linewidth=2)
ax.set_title("Pulsar CP 1919: profil nr 1")
```

```
6 ax.set_xlabel("numer próbki (czas)")
7 ax.set_ylabel("natężenie (j. umowne)")
8 plt.show()
```

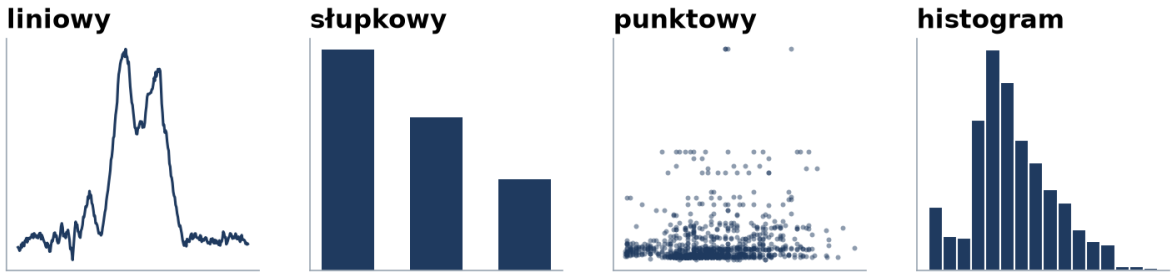
Pierwszy wykres: wynik



Jeden impuls pulsara: natężenie sygnału radiowego w kolejnych chwilach.

Dokładnej skali czasu w danych nie ma: pochodzą z digitalizacji rysunku, a natężenie jest w jednostkach umownych.

Typ wykresu zależy od pytania



Pytanie	Wykres	Metoda
Zmiana w czasie?	liniowy	`ax.plot()`
Porównanie grup?	słupkowy	`ax.bar()`, `ax.barh()`

Pytanie	Wykres	Metoda
Związek dwóch wielkości?	punktowy	`ax.scatter()`
Rozkład wartości?	histogram	`ax.hist()`

Słupkowy, histogram, punktowy

```
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(15, 4))

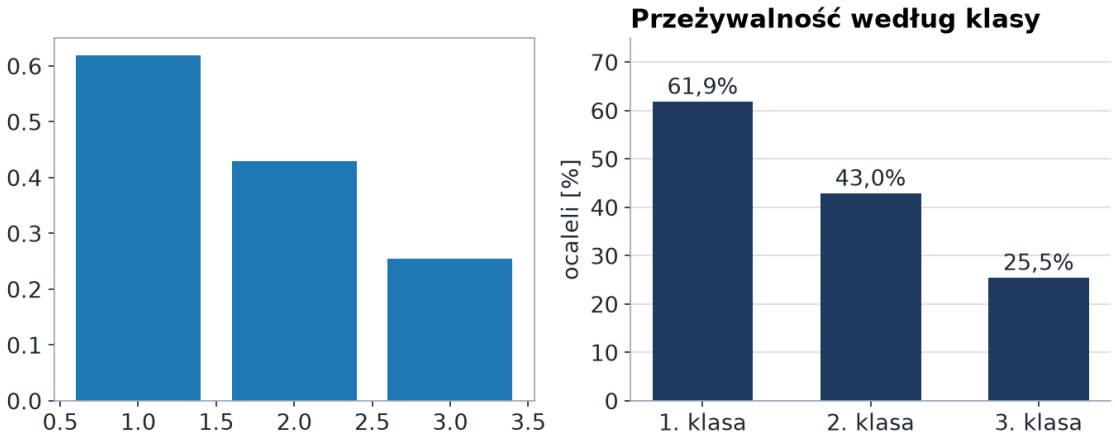
ax1.bar(["1. klasa", "2. klasa", "3. klasa"], [61.9, 43.0, 25.5])

ax2.hist(ages, bins=range(0, 85, 5))           # przedziały co 5 lat

pairs = [(p.age, p.fare) for p in passengers
         if p.age is not None and p.fare is not None]
age, fare = zip(*pairs)
ax3.scatter(age, fare, s=10, alpha=0.5)     # rozmiar, przezroczystość
```

- `bins=` w histogramie zmienia obraz: zbyt szerokie przedziały ukrywają szczegóły, zbyt wąskie dają szum.
- `zip(*pairs)` rozdziela listę par na dwie sekwencje: wieki i opłaty.

Opis wykresu



- **Tytuł** mówi, co widać. **Osie** mają opisy i jednostki.
- **Legenda**, gdy serii jest więcej niż jedna. **Siatka** dyskretna, w tle.
- Słupki zawsze od zera; etykiety wartości tam, gdzie liczba jest ważna.

Opis wykresu w kodzie

```
ax.bar(classes, percents, color="#1F3A5F", width=0.6, zorder=2)
ax.set_title("Przeżywalność według klasy")
ax.set_ylabel("ocaleli [%]")
ax.set_ylim(0, 75)
ax.grid(axis="y", zorder=0)

for i, p in enumerate(percents):
    ax.text(i, p + 1.5, f"{p:.1f}%", ha="center")
```

- `zorder`: kolejność rysowania; większa wartość na wierzchu (siatka za słupkami).
- `ax.text(x, y, text)`: napis w układzie współrzędnych danych.

`plt.style.use("ggplot")` zmienia wygląd wszystkich wykresów naraz; lista stylów: `plt.style.available`.

Zapis do pliku

```
fig.savefig("wykres.png", dpi=300)                # grafika rastrowa
fig.savefig("wykres.pdf")                         # grafika wektorowa
fig.savefig("wykres.png", dpi=150, bbox_inches="tight")
```

Format	Do czego
PNG	prezentacje, strony WWW, e-mail; `dpi` decyduje o ostrości
PDF, SVG	druk, raporty; skalowanie bez utraty jakości

`savefig` wywołujemy **przed** `plt.show()`: po zamknięciu okna figura może być już pusta.

Pokaz w VS Code

- Profil pulsara: `pyplot` i wersja obiektowa obok siebie.
- Przeżywalność według klasy: słupki, etykiety wartości, siatka za słupkami.
- Histogram wieku dla 5, 16 i 80 przedziałów.
- Wiek a opłata: `scatter` z `alpha`.
- Zapis do PNG i PDF; porównanie plików przy powiększeniu.

3.2 Studium przypadku

Kto przeżył katastrofę Titanica?

- Czy zasadę „najpierw kobiety i dzieci” widać w danych?
- Jaką rolę odegrała klasa biletu?
- Czy jedno wynika z drugiego?

Plan analizy: wczytanie → czyszczenie → agregacja → wykresy → eksport → interpretacja.

Wyłącznie narzędziami z kursu: `csv`, słowniki, `Counter`, nasze klasy, NumPy, matplotlib.

Krok 1: wczytanie

```
from pathlib import Path
from passenger_list import PassengerList

DATA_FILE = Path(__file__).resolve().parents[2] / "data" / "titanic3.csv"
RESULTS = Path(__file__).resolve().parent / "results"
RESULTS.mkdir(exist_ok=True)

passengers = PassengerList.from_csv_file(DATA_FILE)
print(passengers)   # <PassengerList: 1309 osób>
```

- Moduły z sesji 2 wykonują całą pracę: konwersję typów, braki jako `None`, walidację.
- `mkdir(exist_ok=True)`: utwórz folder, chyba że już istnieje.

Krok 2: raport braków

```
with open(DATA_FILE, encoding="utf-8", newline="") as f:
    rows = list(csv.DictReader(f))

missing = {col: sum(w[col].strip() == "" for w in rows)
           for col in rows[0]}
for col, n in sorted(missing.items(), key=lambda kv: kv[1], reverse=True):
    if n:
        print(f"{col:<10} {n:5} {n / len(rows):6.1%}")
```

```
body         1188  90.8%
cabin        1014  77.5%
boat          823  62.9%
home.dest     564  43.1%
age           263  20.1%
embarked        2   0.2%
fare            1   0.1%
```

Raport braków liczymy na surowych wierszach: tam brak to pusty napis.

Decyzje o brakach

- **Wiek**: brakuje 20%. Nie zgadujemy; analizy wieku liczymy na 1046 osobach, a brak to osobna grupa „nieznany".
- **Opłata**: jeden brak; 17 biletów za 0 £ to prawdziwe wartości, nie braki.
- **`boat`, `body`**: numer szalupy i numer ciała to **skutek** przeżycia lub śmierci, a nie jego przyczyna.

Zmienna, która koduje wynik, „wyjaśnia" go perfekcyjnie i nie mówi nic. Takich kolumn nie używamy do wyjaśniania przeżycia.

W uczeniu maszynowym to tzw. wyciek danych (data leakage). Braki wieku nie są losowe: brak wieku ma 29% pasażerów 3. klasy, 12% w 1. klasie i 6% w 2. klasie.

Krok 3: agregacja

```python
from collections import Counter

total = Counter((p.pclass, p.sex) for p in passengers)
survivors = Counter((p.pclass, p.sex)
                    for p in passengers if p.survived)
rates = {k: survivors[k] / total[k] for k in total}
```

Kluczem jest **krotka `(pclass, sex)`**: dwa liczniki i wyrażenie słownikowe zastępują tabelę przestawną.

Klasa	Kobiety	Mężczyźni
1	96,5% (n = 144)	34,1% (n = 179)
2	88,7% (n = 106)	14,6% (n = 171)
3	49,1% (n = 216)	15,2% (n = 493)

Kobiety i dzieci

Grupa	Przeżywalność	Liczność
kobiety	72,7%	466
mężczyźni	19,1%	843
dzieci (< 18 lat)	52,6%	154
dorośli (18–59)	39,2%	852
seniorzy (60+)	30,0%	40
wiek nieznany	27,8%	263

Dzieci według klasy: 1. klasa 86,7% (15), 2. klasa 87,9% (33), **3. klasa 36,8% (106)**.

Krok 4: wykresy

```
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(15, 4.6))

bars_class_sex(ax1, rates)
age_histogram(ax2, ages)
age_vs_fare(ax3, passengers)

fig.tight_layout()
fig.savefig(RESULTS / "case_study.png", dpi=200)
plt.show()
```

- Każdy wykres rysuje osobna funkcja, która dostaje swój układ `ax` jako parametr.
- Dekompozycja z sesji 2: `main()` czyta się jak spis treści raportu.

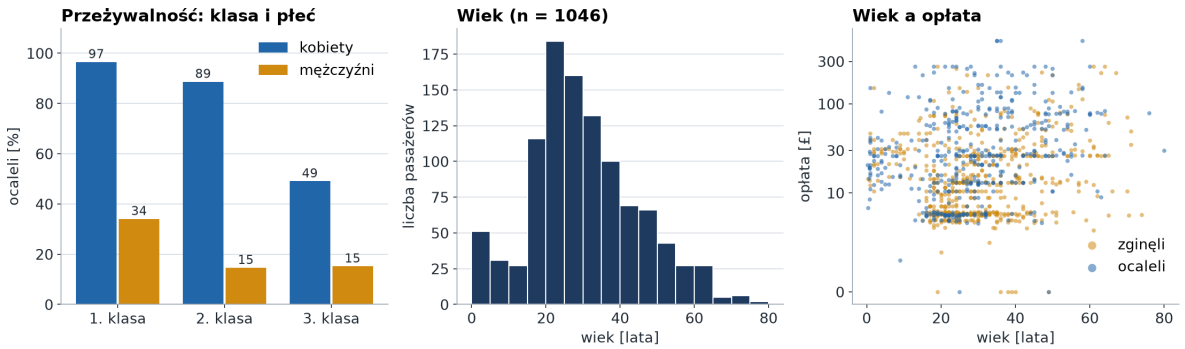
Słupki obok siebie

```
def bars_class_sex(ax, rates):
    x = np.arange(3)
    series = ((-0.2, "female", "kobiety"),
              (0.2, "male", "mężczyźni"))
    for dx, sex, label in series:
        h = [100 * rates[(k, sex)] for k in (1, 2, 3)]
        ax.bar(x + dx, h, width=0.38, label=label)
    ax.set_xticks(x, ["1. klasa", "2. klasa", "3. klasa"])
```

```
    ax.set_ylabel("ocaleli [%]")
    ax.set_title("Przeżywalność: klasa i płeć")
    ax.legend()
```

`x + dx` przesuwa całą tablicę pozycji: słupki kobiet w lewo, mężczyzn w prawo.

Wynik



Na wykresie punktowym oś opłat ma skalę symlog (`ax.set_yscale("symlog", linthresh=10)`): liniowa dla małych wartości (w tym zera), logarytmiczna dla dużych. Bez tego 512 £ spłaszcza wszystkie pozostałe punkty.

Krok 5: eksport

```
file = RESULTS / "survival.csv"
with open(file, "w", encoding="utf-8-sig", newline="") as f:
    writer = csv.writer(f, delimiter=";")
    writer.writerow(["klasa", "plec", "liczba", "ocaleli", "odsetek"])
    for (k, sex), n in sorted(total.items()):
        rate = f"{survivors[(k, sex)] / n:.3f}".replace(".", ",")
        writer.writerow([k, sex, n, survivors[(k, sex)], rate])
```

- Separator `;`, przecinek dziesiętny i `utf-8-sig`: plik otworzy się poprawnie w polskim Excelu.
- Wykres jest już w `results/case_study.png`; tabela i rysunek trafiają do raportu.

Krok 6: interpretacja

- **Płeć** różnicuje najmocniej: przeżyło 72,7% kobiet i 19,1% mężczyzn.
- **Klasa** też: 61,9% / 43,0% / 25,5% dla klas 1 / 2 / 3.
- Zasada „najpierw kobiety i dzieci" działała wyraźnie w 1. i 2. klasie; w 3. klasie przeżyła mniej niż połowa kobiet i co trzecie dziecko.

Klasa biletu jest wskaźnikiem statusu społeczno-ekonomicznego: wiąże się z ceną, wiekiem, położeniem kabiny na statku.

Korelacja a przyczynowość

Udział kobiet w klasach: **44,6% / 38,3% / 30,5%**.

- Część różnicy między klasami wynika więc ze **składu** grup, a nie z samej klasy.
- Dlatego porównujemy w obrębie płci: kobiety w 1. klasie 96,5%, w 3. klasie 49,1%. Różnica zostaje.
- Dane pokazują **związek**. O przyczynach nie mówią: położenie kabin, droga na pokład łodziowy, kolejność dopuszczania do szalup to hipotezy spoza tego pliku.

To jest stratyfikacja, najprostsza ochrona przed zmienną zakłócającą (confounder). Skrajny przypadek to paradoks Simpsona: zależność widoczna w całości może zniknąć lub odwrócić się w podgrupach.

Czego te dane nie obejmują

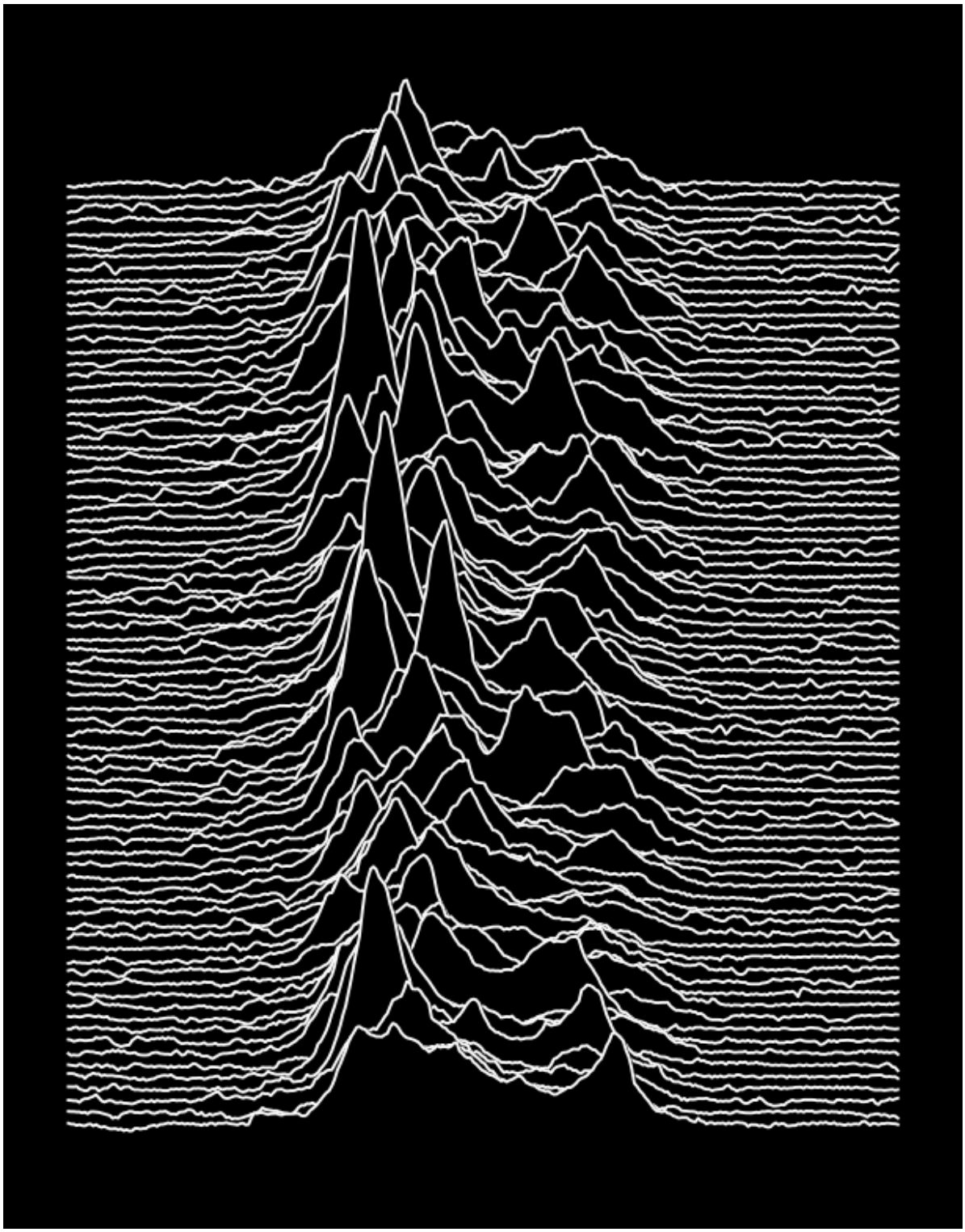
- Zbiór zawiera 1309 pasażerów; **załogi** (ok. 900 osób) w nim nie ma.
- Wieku brakuje u co piątego pasażera, a braki nie rozkładają się równo między klasy.
- Opłata to cena biletu, często wspólnego dla rodziny: nie jest to wydatek jednej osoby.

Każdy wniosek dotyczy **pasażerów z tego zbioru**, z jego brakami.

Pokaz w VS Code

- Studium przypadku od pustego pliku do raportu: kroki 1–6.
- Raport braków na surowych wierszach.
- `Counter` z kluczem `(pclass, sex)`; tabela przeżywalności.
- Trzy wykresy w jednej figurze; funkcje z parametrem `ax`.
- Eksport CSV i PNG; otwarcie wyników w Excelu i w podglądzie.

Laboratorium 5: pulsar CP 1919



- 80 impulsów pierwszego odkrytego pulsara, okres 1,337 s.
- Dane z pracy doktorskiej H. D. Crafta (1970).
- W 1979 r. ten rysunek trafił na okładkę albumu *Unknown Pleasures* zespołu Joy Division.

Odtworzymy go: `np.loadtxt`, pętla, przesunięcie w pionie, `fill_between`, `zorder`, czarne tło.

3.3 Podsumowanie i dalsze kroki

Droga przez kurs

Etap	Co umiemy	Przykład
Typy i kolekcje	opisać dane	pasażer jako słownik
Sterowanie	podejmować decyzje i powtarzać	grupa wiekowa, pętla po pasażerach
Funkcje i moduły	dzielić problem na części	`load()`, `age_group()`
Obiekty	łączyć dane z zachowaniem	`Passenger`, `PassengerList`
Biblioteki	korzystać z gotowych narzędzi	`statistics`, NumPy, matplotlib

Wszystko jest obiektem

```
"allen".upper()                    # sesja 1: metoda napisu
passengers.append(p)               # sesja 1: metoda listy
p.age_group()                      # sesja 2: metoda naszej klasy
passengers.filter(lambda p: ...)   # sesja 2: funkcja jako argument
age_array[age_array < 18].mean()   # sesja 3: metoda tablicy NumPy
ax.set_title("Przeżywalność")      # sesja 3: metoda układu wsp.
```

Ten sam zapis z kropką od pierwszego do ostatniego wykładu. Biblioteki to klasy i funkcje napisane przez innych, według tych samych zasad.

Asystenci AI: krytycznie

Wygenerowany kod, który działa bez błędu:

```
ages = [float(w["age"] or 0) for w in rows]
sum(ages) / len(ages)            # 23.877...
```

Poprawny wynik to **29,88**: 263 braki wieku policzono jak noworodki.

- Kod, który się uruchamia, nie musi liczyć tego, o co pytamy.
- Sprawdzamy na liczbach, które znamy: liczba wierszy, jeden ręcznie policzony przypadek.
- Pytamy o braki, jednostki i typy; nie uruchamiamy kodu, którego nie rozumiemy.

Nawiązanie do sesji 1: AI przesuwa wąskie gardło z pisania kodu na jego sprawdzanie. Ten przykład to dokładnie ten rodzaj błędu: brak wyjątku, wiarygodna liczba, zły wniosek.

Co dalej?

- **Analiza danych: pandas.** Tabela jako obiekt; nasza agregacja w jednej linii:

```
import pandas as pd
df = pd.read_csv("data/titanic3.csv")
df.groupby(["pclass", "sex"])["survived"].mean()
```

- **Automatyzacja**: `pathlib` i pętla po plikach, arkusze Excela (`openpyxl`), raporty PDF.
- **Proste aplikacje**: Streamlit, czyli interfejs w przeglądarce napisany w czystym Pythonie.

Lektura: E. Matthes, Python. Instrukcje dla programisty; J. VanderPlas, Python Data Science. pandas pokazujemy tylko jako kierunek: wszystko, co robi `groupby`, umiemy już zapisać słownikiem i `Counter`. Dzięki temu wiadomo, co pandas robi pod spodem.

Laboratorium

Grupa	Terminy
zarz/z/lic/III/ćw NS/AB	13.12.2026, 30.01.2027
zarz/zclouda/lic/III/ćw/ABb	19.12.2026, 16.01.2027, 30.01.2027
zarz/zclouda/lic/III/ćw/ABa	19.12.2026, 16.01.2027, 30.01.2027

- 5 bloków po 3 godziny; każdy zaczyna się krótką powtórką.
- Własny komputer z Pythonem i VS Code albo Google Colab.
- Kod z wykładów jest w repozytorium kursu: wersje gotowe (`final/`) i szkielety (`starter/`).

Podsumowanie

- `float` do pomiarów i statystyki, `Decimal` z `ROUND_HALF_UP` do pieniędzy.
- `statistics`: miary opisowe; przy danych skośnych mediana zamiast średniej.
- NumPy: jednorodne tablice, działania wektorowe, maski, `nan`.
- matplotlib: `fig, ax = plt.subplots()`, wykres dobrany do pytania, opisany i zapisany.
- Analiza danych kończy się **interpretacją**, nie wykresem.

Do zobaczenia na laboratorium.